



kubernetes

Kubernetes

L'orchestrateur de containers de référence

Sommaire

1. Introduction à Kubernetes

- Virtualisation / conteneurisation - Docker
- Solutions d'orchestration de conteneurs - Docker Swarm, Kubernetes...
- Notions de conteneurs statefull - stateless
- Rappels sur les fichiers YAML

2. Architecture Kubernetes

- Nodes et services d'un cluster Kubernetes
- Addons pour le cluster

Sommaire

3. Installation d'une plateforme

- Installation du client kubectl
- Installation Minikube, Microk8s
- Installation d'un cluster
- Utilisation sur le Cloud

4. Objets Kubernetes

- Pods
- Services
- Volumes persistents
- Deployments
- ReplicaSet
- StatefulSet
- DaemonSet
- Jobs
- Cronjob

5. Exploitation Kubernetes

- Labels et annotations
- Affinité, anti-affinité
- Health-checks
- Configmap
- Secrets

6. Kubernetes en production

- Frontal administrable Ingress
- Limitation des ressources - requests et limits
- Namespaces, contextes et quotas
- HPA - Horizontal Pod Autoscaling
- RBAC - Gestion des accès
- Mode maintenance - drain, cordon
- Stratégie de mise à jour sans interruption

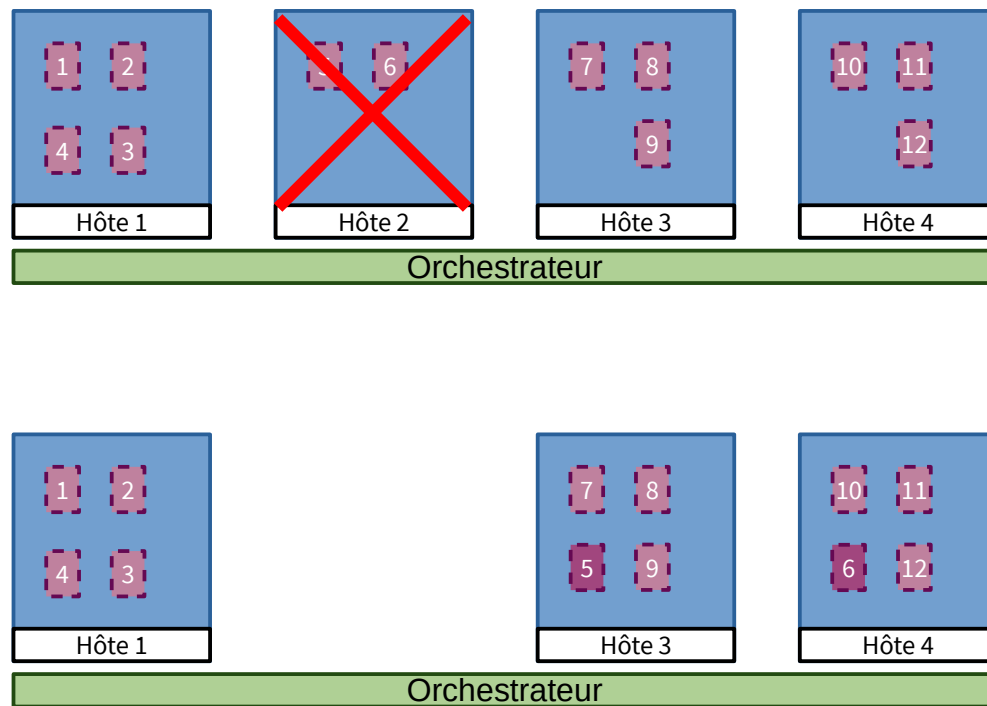
Orchestrateurs de containers

Mettre en production des containers

Containers - Orchestrateurs

Rôles des orchestrateurs de containers

- Centraliser la gestion des containers
- Gérer la tolérance à la panne des hôtes
- Gérer la répartition de charge (scaling)
- Gérer le déploiement / mise à jour des containers
- Gérer l'accessibilité des containers
- Gérer la connectivité des containers entre les hôtes

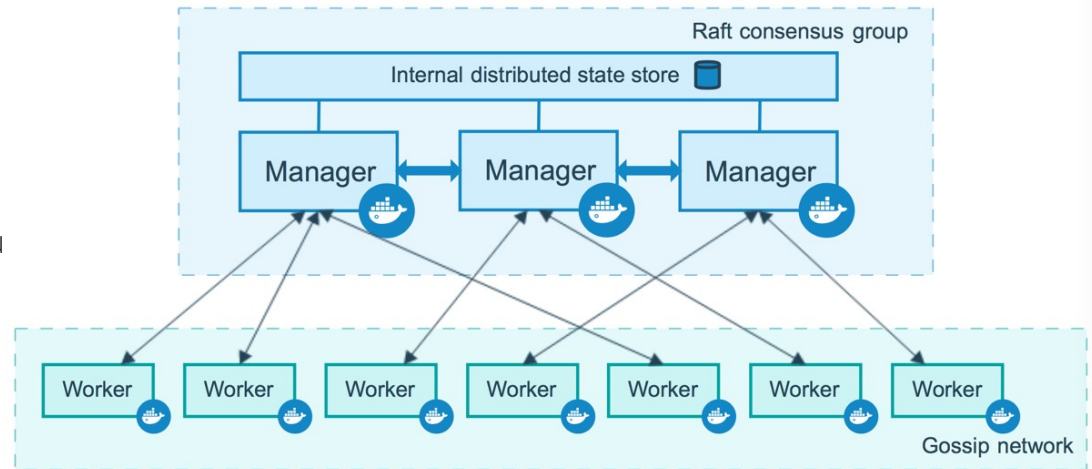


Containers - Orchestrateurs

Docker Swarm

- Intégré à Docker CE
- Simple à apprendre
- Utilise la commande docker CLI
 - `docker swarm init`
 - `docker swarm join`
- Même syntaxe que `docker run`
- Service : désigne un container pouvant disposer de plusieurs réplicas
- Routage du trafic entre les nodes
- Déploiement des applications par un fichier YAML Docker Compose

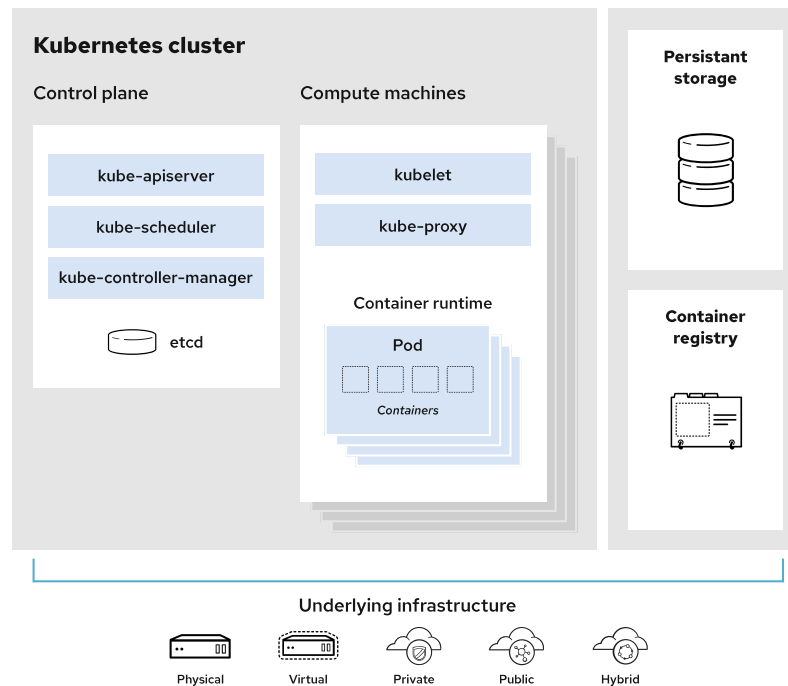
- Nodes Manager : gestion des workers
- Nodes Worker : exécution des containers



Docker - Orchestrateurs

Kubernetes

- Développé par Google
- Open-source
- Largement déployé
- Architecture complexe
- Indépendant de la technologie de conteneurisation
- POD : containers partageant le même stockage
- kubectl
- deployment, service, cronjob, persistant storage, config map, ...



Containers Statefull - Stateless

Container Stateless

Container dont aucune donnée ne nécessite un stockage permanent. Effectue essentiellement des traitements. Les données à stocker sont éventuellement transmises à un autre container ou à un autre service.

Le container d'une application **PHP** ne sera **stateless** que s'il ne stocke aucune donnée à conserver après suppression. Il n'a pas à conserver un **état** des données.

Container Statefull

Container nécessitant de conserver des données de manière persistante. Il doit faire appel à du stockage persistant afin de retrouver les données stockées en cas de recreation.

Typiquement, un container de base de données sera **statefull** : il devra conserver l'état des données.

Rappels sur les fichiers YAML



Kubernetes

Caractéristiques

- Utilisé pour structurer des fichiers de configuration (Kubernetes, Docker Compose, Ansible...)
- Lecture facile grâce à l'obligation d'indentation pour l'imbrication des données.
- Fichier texte utilisant un encodage **UTF-8**.
- **L'indentation** doit obligatoirement être effectuée avec le symbole espace (1 minimum) et pas avec le symbole ~~tabulation~~.
- L'utilisation d'un éditeur texte approprié (vim, vscode, notepad++...) peut être nécessaire pour faciliter les manipulations :
 - **autoindentation** : le retour à la ligne permet de conserver le niveau d'indentation
 - **utilisation des espaces** : la touche tabulation crée des espaces plutôt que des tabulations, le nombre d'espaces est paramétrable. La tabulation peut être effectuée sur plusieurs lignes simultanément.
 - **repli des variables** : le symbole > permet de replier les clés enfants d'une clé pour faciliter la lecture.
 - **trait d'alignement vertical** : un trait vertical apparaît pour visualiser l'alignement des caractères sur les différents niveaux d'indentation.

Kubernetes

Structures du fichier

- Un fichier YAML peut contenir plusieurs documents YAML. Ils sont alors séparés par ---.
- Le première séquence de tirets n'est pas nécessaire.
- Un document YAML est un ensemble de clés stockant des valeurs de différents types.

```
---  
fichier:  
  nom: document1  
  date: 2022-11-13  
---  
fichier:  
  nom: document2  
  date: 2022-11-14  
---  
fichier:  
  nom: document3  
  date: 2022-11-15
```

Kubernetes

Valeur scalaire : une valeur simple, le type n'a pas à être identifié.

```
nom: document1
date de création: 2022-11-13
"date de modification": 2022-11-14
taille: 4096
compressé: false
label: null
descriptif court: "Le premier document exemple"
descriptif long: |
    Ceci est un fichier montrant le fonctionnement du format YAML.
    Ce descriptif se code sur plusieurs lignes grâce au symbole |.
    Le retour à la ligne ne sera pas indenté quelque soit l'indentation du bloc de texte.
descriptif long sans retour à la ligne: >
    Dans ce descriptif, les retours à la ligne
    se traduiront par un espace.

    Il faut une ligne vide pour créer un retour à la ligne.
```

Kubernetes

Séquence : liste de valeurs pouvant être de type différents

```
composants:
```

- processeurs
- mémoire vive
- disque dur

Kubernetes

Mapping : une ou plusieurs associations clés / valeurs

```
personne:  
  nom: MASSET  
  prenom: Anthony  
  adresse email: anthony@masset.name
```


Séquence de mappings

```
personnes:  
  - nom: MASSET  
    prenom: Anthony  
    adresse_email: anthony@masset.name  
  - nom: MASSET  
    prenom: Alyssa  
    adresse_email: alyssa@masset.name  
  - nom: MASSET  
    prenom: Eline  
    adresse_email: eline@masset.name
```

Kubernetes

Les commentaires sont précédés d'un #.

```
# Ceci est un commentaire  
nom: MASSET
```

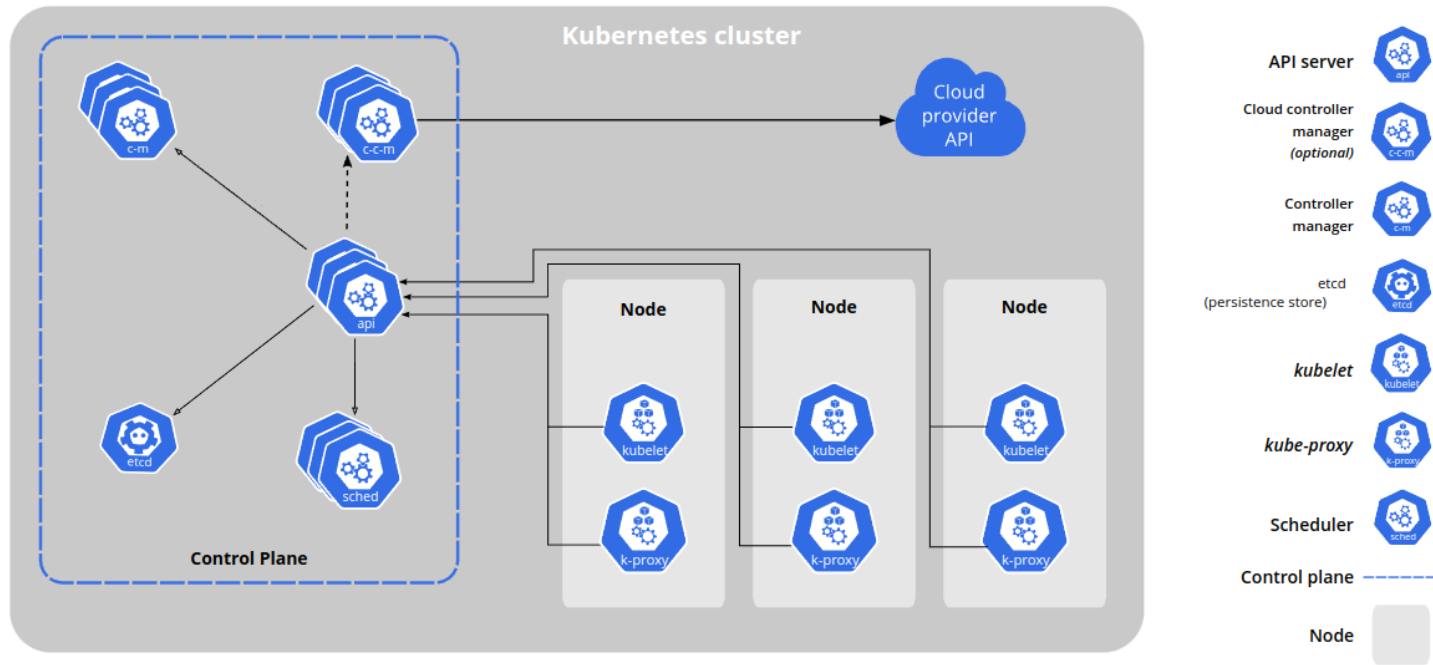
Il est possible d'intégrer des valeurs au format json dans un document yaml.

```
# Exemple d'une séquence au format JSON  
composants: [processeur, mémoire vive, disque dur]  
  
# Exemple d'un mapping au format JSON  
personne: {"nom": "MASSET", "prénom": "Anthony", "adresse email": "anthony@masset.name"}
```

Architecture Kubernetes

Kubernetes

Composants Kubernetes



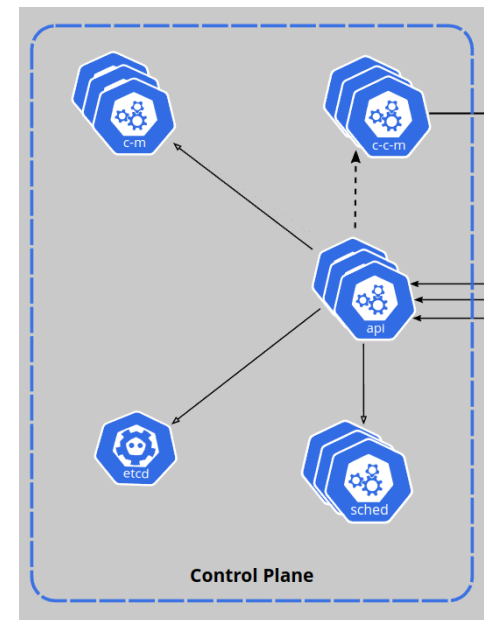
Kubernetes

Control Plane

Le **Control Plane** est exécuté sur au minimum un **node** (machine) qui sera considéré comme **master**.

Il prend les décisions concernant le cluster dans sa globalité comme par exemple :

- Démarrage des pods sur un node worker
- Adéquation du nombre de répliquas



Kubernetes

Services Kubernetes - kube-apiserver

- Expose l'API sur le front.
- Utilisé par les applications tierces et les commandes CLI (`kubectl`, `curl`...) pour accéder au cluster.
- Possibilité de mise à l'échelle horizontale en ajoutant plusieurs instances et en répartissant les requêtes.



Kubernetes

Services Kubernetes - etcd

Service de stockage de type clé-valeur permettant de stocker les données du cluster (configuration...).

Faire une sauvegarde des données etcd est une bonne pratique.

```
# Création d'un snapshot
ETCDCTL_API=3 etcdctl --endpoints $ENDPOINT snapshot save snapshotdb

# Restauration d'un snapshot
ETCDCTL_API=3 etcdctl --endpoints 10.2.0.9:2379 snapshot restore snapshotdb
```

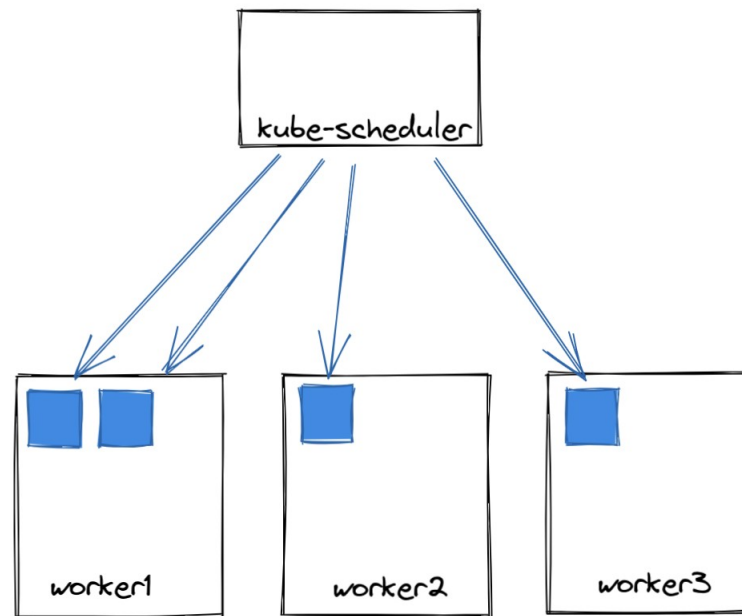
Kubernetes

Services Kubernetes – kube-scheduler

S'occupe de l'assignation des pods aux nodes worker.

Prend des décisions d'affectation en regardant les paramètres suivants :

- Ressources demandées
- Contraintes matérielles, logicielles
- Règles d'affinités et d'anti-affinités
- Localisation des données
- ...



Services Kubernetes – kube-controller-manager

Ce service fait tourner des processus de type "controller" :

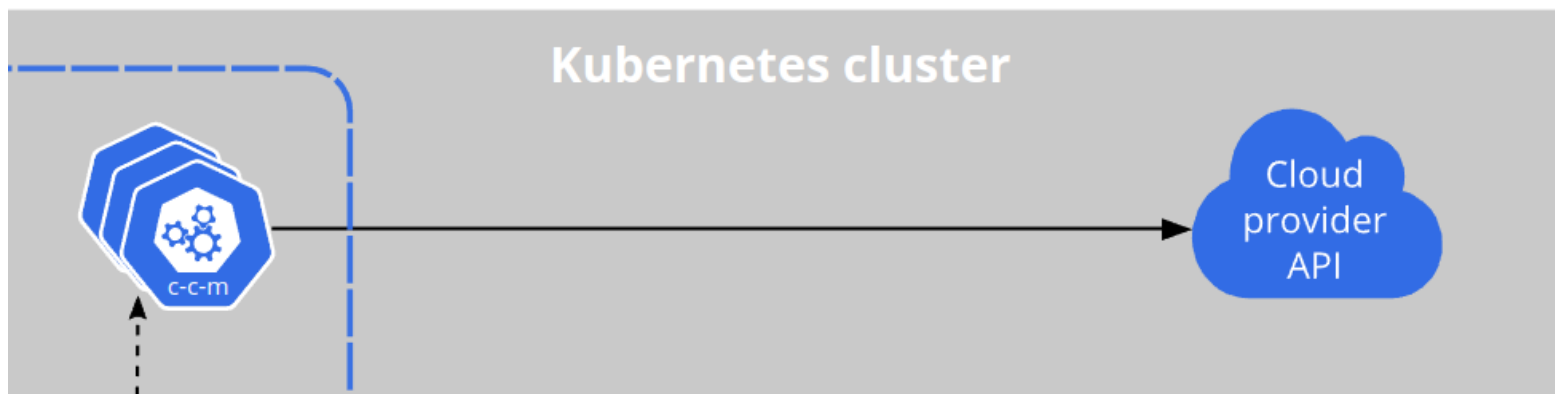
- **Node Controller** : notifie le cluster quand des noeuds s'arrêtent.
- **Job Controller** : contrôle l'exécution des jobs ponctuels
- **EndpointSlice controller** : contrôle la création des points de connexion réseau notamment pour les services.
- **ServiceAccount controller** : créer les comptes de services nécessaire pour d'autres namespaces.

Kubernetes

Services Kubernetes – cloud-controller-manager

Permet l'interaction avec les API d'un provider Cloud.

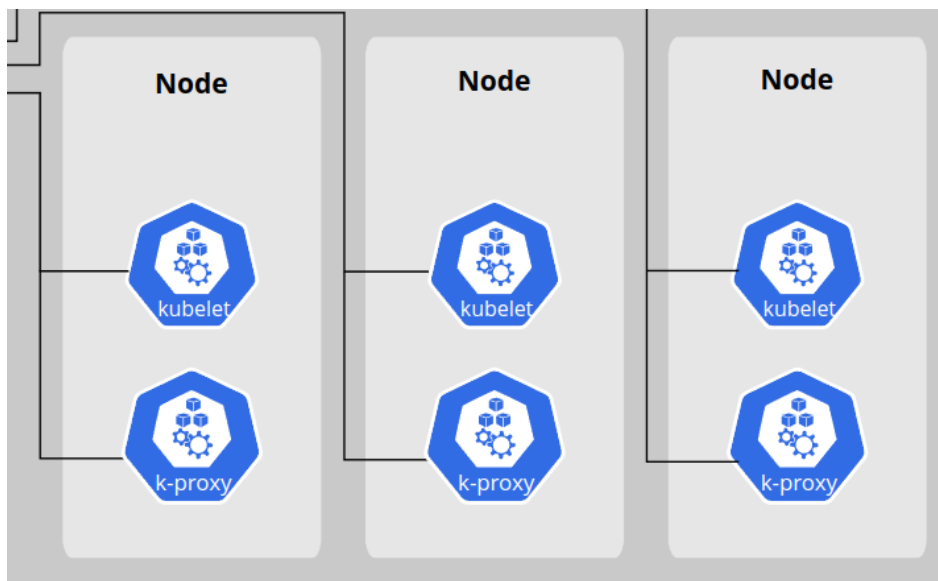
N'est pas utilisé sur une installation hors Cloud.



Kubernetes

Nodes Worker

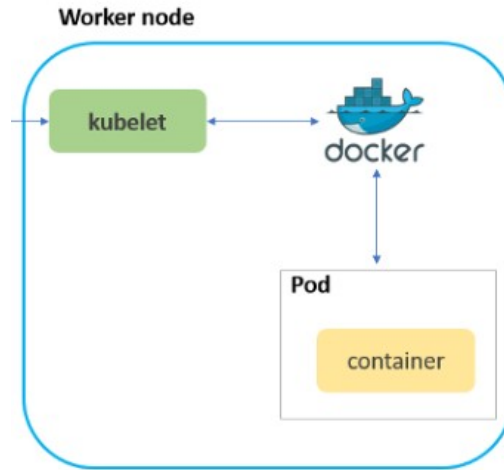
Les nodes hébergent les pods permettant de faire tourner les applications.



Kubernetes

Services Kubernetes – kubelet

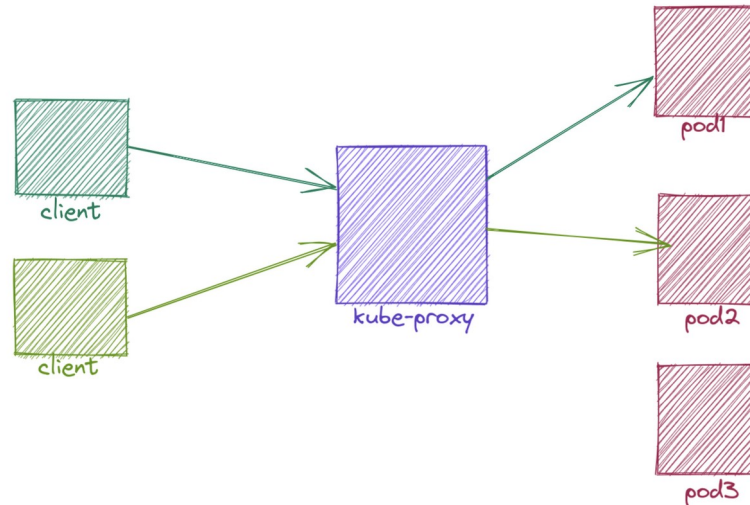
- Il permet de s'assurer que les containers s'exécutent correctement dans les pods.
- Il s'assure que les caractéristiques transmises par différents mécanismes soient respectées.



Kubernetes

Services Kubernetes – kube-proxy

- Il s'agit d'un proxy réseau permettant de faire tourner les services.
- Le trafic passe par ce service avant de toucher le pod concerné.



Kubernetes

Services Kubernetes – Moteur de conteneurisation

- Le node doit permettre l'exécution de conteneurs, il doit alors communiquer avec un moteur de conteneurisation (`container runtime`).
- Kubernetes est compatible avec :
 - Docker Engine
 - Containerd
 - CRI-O
 - Mirantis Container Runtime



Addons pour le cluster

Kubernetes

Addons pour le cluster

Kubernetes peut utiliser divers addons pour gérer le cluster.

Liste des addons actuellement supportés :

<https://kubernetes.io/docs/concepts/cluster-administration/addons/>

Networking and Network Policy

- **ACI** provides integrated container networking and network security with Cisco ACI.
- **Antrea** operates at Layer 3/4 to provide networking and security services for Kubernetes, leveraging Open vSwitch as the networking data plane.
- **Calico** is a networking and network policy provider. Calico supports a flexible set of networking options so you can choose the most efficient option for your situation, including non-overlay and overlay networks, with or without BGP. Calico uses the same engine to enforce network policy for hosts, pods, and (if using Istio & Envoy) applications at the service mesh layer.
- **Canal** unites Flannel and Calico, providing networking and network policy.
- **Cilium** is a networking, observability, and security solution with an eBPF-based data plane. Cilium provides a simple flat Layer 3 network with the ability to span multiple clusters in either a native routing or overlay/encapsulation mode, and can enforce network policies on L3-L7 using an identity-based security model that is decoupled from network addressing. Cilium can act as a replacement for kube-proxy; it also offers additional, opt-in observability and security features.
- **CNI-Genie** enables Kubernetes to seamlessly connect to a choice of CNI plugins, such as Calico, Canal, Flannel, or Weave.
- **Contiv** provides configurable networking (native L3 using BGP, overlay using vxlan, classic L2, and Cisco-SDN/ACI) for various use cases and a rich policy framework. Contiv project is fully [open sourced](#). The **installer** provides both kubeadm and non-kubeadm based installation options.
- **Contrail**, based on **Tungsten Fabric**, is an open source, multi-cloud network virtualization and policy management platform. Contrail and Tungsten Fabric are integrated with orchestration systems such as Kubernetes, OpenShift, OpenStack and Mesos, and provide isolation modes for virtual machines, containers/pods and bare metal workloads.
- **Flannel** is an overlay network provider that can be used with Kubernetes.
- **Knitter** is a plugin to support multiple network interfaces in a Kubernetes pod.
- **Multus** is a Multi plugin for multiple network support in Kubernetes to support all CNI plugins (e.g. Calico, Cilium, Contiv, Flannel). In addition to SRIOV, DPDK, OVS-DPDK and VPP based workloads in Kubernetes.
- **OVN-Kubernetes** is a networking provider for Kubernetes based on **OVN (Open Virtual Network)**, a virtual networking implementation that came out of the Open vSwitch (OVS) project. OVN-Kubernetes provides an overlay based networking implementation for Kubernetes, including an OVS based implementation of load balancing and network policy.
- **Nodeus** is an OVN based CNI controller plugin to provide cloud native based Service function chaining(SFC).

- **NSX-T** Container Plug-in (NCP) provides integration between VMware NSX-T and container orchestrators such as Kubernetes, as well as integration between NSX-T and container-based CaaS/PaaS platforms such as Pivotal Container Service (PKS) and OpenShift.
- **Nuage** is an SDN platform that provides policy-based networking between Kubernetes Pods and non-Kubernetes environments with visibility and security monitoring.
- **Romana** is a Layer 3 networking solution for pod networks that also supports the **NetworkPolicy** API.
- **Weave Net** provides networking and network policy, will carry on working on both sides of a network partition, and does not require an external database.

Service Discovery

- **CoreDNS** is a flexible, extensible DNS server which can be [installed](#) as the in-cluster DNS for pods.

Visualization & Control

- **Dashboard** is a dashboard web interface for Kubernetes.
- **Weave Scope** is a tool for graphically visualizing your containers, pods, services etc. Use it in conjunction with a [Weave Cloud account](#) or host the UI yourself.

Infrastructure

- **KubeVirt** is an add-on to run virtual machines on Kubernetes. Usually run on bare-metal clusters.
- The **node problem detector** runs on Linux nodes and reports system issues as either [Events](#) or [Node conditions](#).

Kubernetes

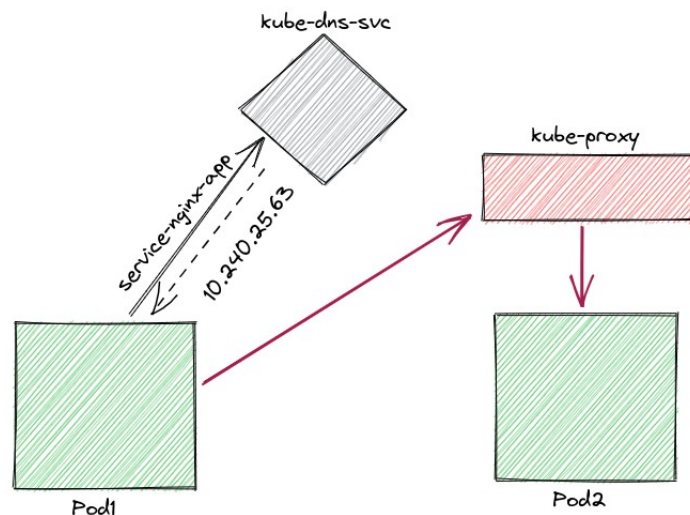
Addons – DNS

Addon obligatoire sur un cluster !

C'est un serveur DNS qui résout des noms pour les services de Kubernetes. Les ressources peuvent être contactées par un nom plutôt que par une adresse IP.

Il est utilisé par les containers démarrés par Kubernetes.

Installé automatiquement lors de la mise en place du Control Plane.

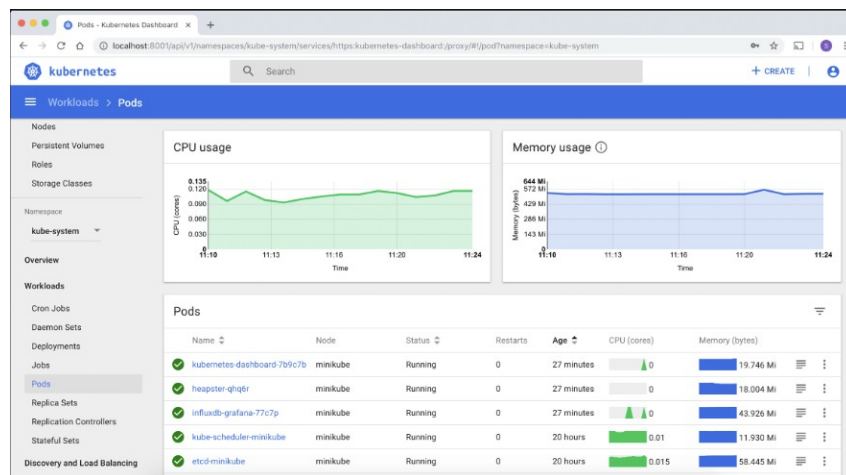


Kubernetes

Addons – Dashboard

Il s'agit d'une interface graphique permettant de gérer de manière plus conviviale pour gérer les ressources Kubernetes.

```
# Installation du dashboard  
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.6.1/aio/deploy/recommended.yaml
```



Kubernetes

Addons – Container Resource Monitoring

Permet de surveiller les ressources consommées à l'aide de métriques.

```
# Installation du serveur de métriques
```

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Nodes 4 items Search...									
Name ▾	CPU ▾	Memory ▾	Disk ▾	Taints ▾	Roles ▾	Version ▾	Age ▾	Conditions ▾	⋮
kmaster				2	control-plane	v1.25.3	194d	Ready	⋮
knode1				0		v1.25.3	194d	Ready	⋮
knode2				0		v1.25.3	194d	Ready	⋮
knode3				0		v1.25.3	194d	Ready	⋮

Installation

Kubernetes

Kubernetes peut être déployé de différentes manières :

- Outil **kubeadm** : permet de bootstraper le cluster
- **Minikube** : pour déployer en mono-node sur Windows
- **Microk8s** : pour déployer en mono-node et multi-node sur toutes les plateformes
- **Kubespray** : ensemble de scripts Ansible
- Kubernetes Managé sur le **Cloud** (AWS, GCP...)

L'outil **kubectl** est essentiel pour interagir avec Kubernetes.

Kubernetes

Installation de kubectl

Récupération du numéro de la dernière version stable :

<https://storage.googleapis.com/kubernetes-release/release/stable.txt>

Sur Windows :

```
curl.exe -LO "https://dl.k8s.io/release/v1.25.4/bin/windows/amd64/kubectl.exe"
```

Sur Linux :

```
# Toutes versions
curl -LO https://storage.googleapis.com/kubernetes-release/release/v1.25.4/bin/linux/amd64/kubectl

# Ubuntu
snap install kubectl --classic
```

Configuration de kubectl

Il suffit de récupérer le fichier de configuration du master (initialisé par kubeadm init).

Sur Linux, il doit être placé par défaut dans `~/.kube/config`.

Sur Windows, il doit être placé dans `%USERPROFILE%\.kube/config`.

Le paramètre `--kubeconfig` permet de spécifier le fichier de configuration à utiliser.

La variable d'environnement `KUBECONFIG` permet de spécifier le fichier de configuration à utiliser.

```
kubectl --kubeconfig ~/.kube/config get pods
```

```
export KUBECONFIG=$HOME/.kube/config  
kubectl get pods
```

Kubernetes

Autocomplétion de kubectl

L'autocomplétion permet de faciliter l'utilisation de la commande.

Kubectl completion permet de générer le code d'autocomplétion pour différents systèmes.

```
kubectl completion bash  
echo 'source <(kubectl completion bash)' >> ~/.bashrc
```

```
kubectl completion powershell | Out-String | Invoke-Expression  
kubectl completion powershell >> $PROFILE
```




Kubernetes

MicroK8S

- Solution développée par Canonical, sponsor de la distribution Ubuntu et très actif dans le domaine du Cloud.
- Destiné à être déployée aussi bien sur les postes de développeurs que sur des serveurs pour la mise en production.
- Permet d'alléger la maintenance.
- Version Windows : s'appuie sur Hyper-V ou Virtualbox et Multipass
- Version GNU/Linux : installable par snapcraft
- Version MacOS

Kubernetes

MicroK8S

```
microk8s status  
microk8s enable dashboard dns registry istio  
microk8s kubectl get pods  
microk8s stop  
microk8s start
```

```
# Création du cluster sur le master  
microk8s add-node  
# Sur le node à ajouter  
microk8s join 192.168.1.230:25000/92b2db237428470dc4fcfc4ebbd9dc81/2c0cb3284b05
```

Kubernetes



minikube

Kubernetes

Minikube

- Permet de faciliter l'installation mono-node de Kubernetes.
- Destiné aux développeurs.

Installation

- Récupération du binaire à partir du site officiel : <https://minikube.sigs.k8s.io/docs/start/>

```
minikube start  
minikube kubectl -- get nodes
```

Installation d'un cluster par kubeadm

Kubernetes

Installation de `kubeadm`, de `kubelet` et de `kubectl`

- `kubeadm` permet d'administrer et installer un cluster Kubernetes.
- `kubelet` permet de gérer l'interaction avec les containers (démarrage, arrêt...)
- `kubectl` permet d'interagir avec des clusters Kubernetes.

Kubernetes

```
# Configure persistent loading of modules
sudo tee /etc/modules-load.d/k8s.conf <<EOF
overlay
br_netfilter
EOF

# Load at runtime
modprobe overlay
modprobe br_netfilter

# Ensure sysctl params are set
tee /etc/sysctl.d/kubernetes.conf<<EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF

# Reload configs
sysctl --system

# Install required packages
apt install -y curl gnupg2 software-properties-common apt-transport-https ca-certificates

# Add Docker repo
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"

# Install containerd
apt update
apt install -y containerd.io
```

```
# Configure containerd and start service
su -
mkdir -p /etc/containerd
containerd config default > /etc/containerd/config.toml

# remplacer une variable dans le fichier de conf SystemdCgroup = true
sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml

# restart containerd
systemctl restart containerd
systemctl enable containerd
systemctl status containerd
```

Installation d'un container runtime

Kubernetes

Installation des composants Kubernetes

```
apt-get update
apt-get install -y apt-transport-https ca-certificates curl

curl -fsSLo /usr/share/keyrings/kubernetes-archive-keyring.gpg https://packages.cloud.google.com/apt/doc/apt-key.gpg

echo "deb [signed-by=/usr/share/keyrings/kubernetes-archive-keyring.gpg] https://apt.kubernetes.io/ kubernetes-xenial main" | sudo tee
/etc/apt/sources.list.d/kubernetes.list

apt-get update
apt-get install -y kubelet kubeadm kubectl
apt-mark hold kubelet kubeadm kubectl
```

Kubernetes

Désactivation du swap pour tous les nodes

```
swapoff -a  
vim /etc/fstab
```

Kubernetes

Initialisation du cluster sur le master (Control Plane)

```
kubeadm init --pod-network-cidr=10.244.0.0/16
# Copie du fichier de conf pour kubectl
mkdir /root/.kube
cp /etc/kubernetes/admin.conf /root/.kube/config

# Déploiement du pod réseau
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml

# Récupération d'un token pour la jonction des workers
kubeadm token create --print-join-command
```

Kubernetes

Jonction d'un worker

```
kubeadm join 192.168.1.100:6443 --token 0dt0kt.h4i71m34tbfqup83 --discovery-token-unsafe-skip-ca-verification
```

Objets Kubernetes

Kubernetes

Pods

- Groupe d'un ou de plusieurs containers ayant un **stockage partagé**, des ressources réseaux et une seule spécification sur le fonctionnement des containers.
- Plus petite unité déployable.
- Généralement créés par l'intermédiaire de Deployments, de Jobs, de StatefulSet et de DaemonSet.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

```
kubectl apply -f nginx.yml
```

Kubernetes

Plusieurs containers par Pod ?

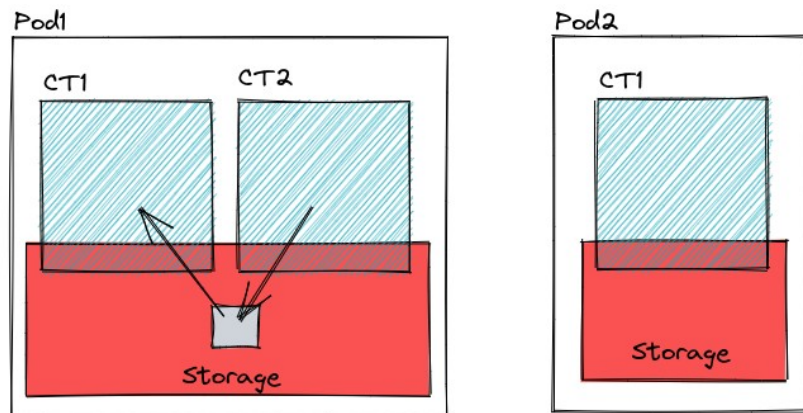
Généralement 1 seul container par Pod.

Peuvent accéder au même stockage (volumes).

Utilisent le même réseau (localhost entre eux).

Cas de plusieurs containers par Pod :

- 1 container présente des données au client
- 1 container met à jour ces données



Kubernetes

Templates de Pods

- Permet la création des Pods suivant les spécifications du template.
- Utilisables dans Deployments, ReplicaSet, Jobs et DaemonSets.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello
spec:
  template:
    # This is the pod template
    spec:
      containers:
        - name: hello
          image: busybox:1.28
          command: ['sh', '-c', 'echo "Hello, Kubernetes!" && sleep 3600']
          restartPolicy: OnFailure
    # The pod template ends here
```


Services

kubernetes

Services

Les services décrivent comment accéder aux Pods par le réseau.

Ils s'appuient sur le service kube-proxy.

```
anthony@superpc:~$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
avaform-convert-service	NodePort	10.110.139.41	<none>	5000:30100/TCP	157d
avaform-service	NodePort	10.100.216.132	<none>	8094:30094/TCP	157d
cpe-service	NodePort	10.105.89.175	<none>	8089:30089/TCP	212d
ddns-service	NodePort	10.111.108.184	<none>	8062:30062/TCP	18d
dgacagents-mysql-dev-service	NodePort	10.98.116.191	<none>	3306:30063/TCP	211d
dgacagents-mysql-service	NodePort	10.97.152.163	<none>	3306:30059/TCP	212d
dgacagents-php-dev-service	NodePort	10.96.17.135	<none>	80:30099/TCP	211d
dgacagents-service	NodePort	10.98.206.13	<none>	80:30098/TCP	212d
dokuwiki-service	NodePort	10.96.193.148	<none>	8050:30050/TCP	211d
easyappointments-service	NodePort	10.99.127.25	<none>	80:30113/TCP	65d
fake-payments-service	NodePort	10.111.21.134	<none>	80:30913/TCP	43h
guacamole-service	NodePort	10.99.139.246	<none>	8080:30111/TCP	211d
guacd-service	ClusterIP	10.100.75.30	<none>	4822/TCP	211d
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	212d
matomo-service	NodePort	10.100.36.127	<none>	8070:30070/TCP	37d
matuvu-app-service	NodePort	10.109.229.144	<none>	8097:30097/TCP	212d
mgm-app-dev-mysql-service	NodePort	10.102.224.128	<none>	3358:30058/TCP	211d
mgm-app-dev-service	NodePort	10.108.250.236	<none>	8096:30096/TCP	211d
mgm-app-mysql-service	NodePort	10.98.52.56	<none>	3357:30057/TCP	212d
mgm-app-service	NodePort	10.98.205.16	<none>	8095:30095/TCP	212d
phicalendar-service	NodePort	10.99.172.158	<none>	80:30114/TCP	211d
site-formation-service	NodePort	10.107.93.222	<none>	8088:30088/TCP	212d
waf-backoffice-php-service	NodePort	10.111.169.19	<none>	80:30093/TCP	208d

Types de services

ClusterIP : le service est accessible sur une IP interne du cluster. Aucun port n'est associé aux IP des nodes. Cela sert uniquement aux communications inter-pods.

Nodeport : expose un port statique (30000 - 32767) sur chaque node pour accéder au pod rattaché

LoadBalancer : expose le service à l'extérieur par un répartiteur de charge d'un fournisseur de Cloud. Le lien est fait automatiquement entre le répartiteur et les service nodeport et ClusterIP.

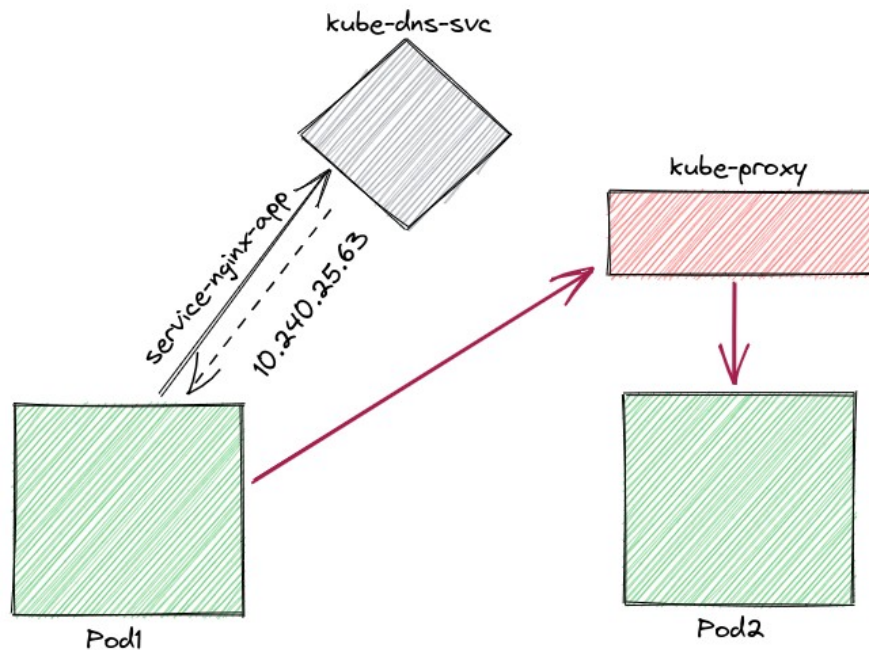
ExternalName : permet simplement de définir un alias au nom de service. Il doit être au format d'un nom DNS qui pourra être consulté depuis l'extérieur.

Résolution DNS

Le service est déclaré avec un nom qui est enregistré automatiquement dans le service DNS intégré au cluster Kubernetes.

Cela permet aux Pods de joindre l'IP du cluster en utilisant le nom.

Le **mapping** vers le Pod est fait automatiquement et ne nécessite aucune intervention.

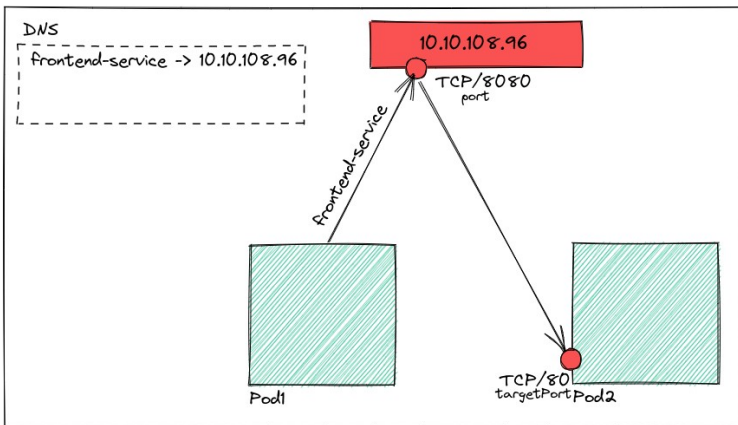


Kubernetes

Service ClusterIP

Le container écoute sur un port (**targetPort**), le service rend accessible sur une IP interne du cluster un port (**port**) redirigeant vers le port du container (**targetPort**).

Node Kubernetes

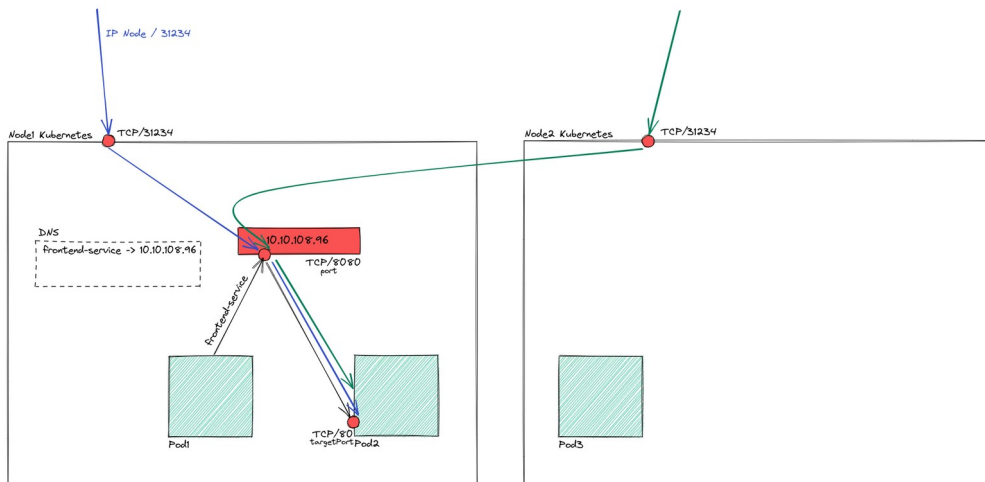


```
apiVersion: v1
kind: Service
metadata:
  name: service-frontend
spec:
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 80
```

Kubernetes

Service NodePort

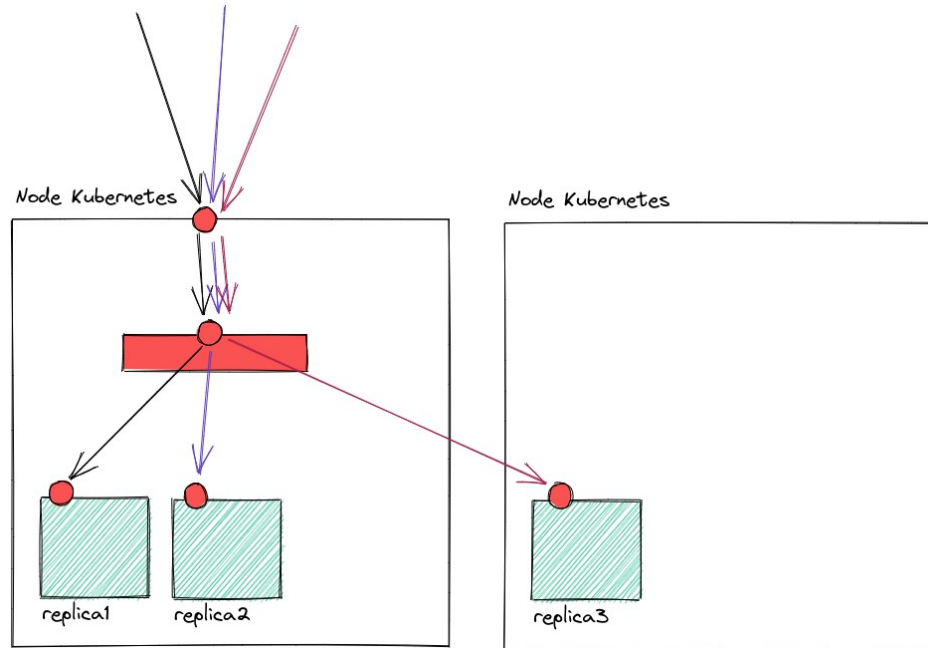
Un port est placé en écoute sur chaque adresse de chaque node pour permettre un accès depuis l'extérieur du cluster. Ce port doit être compris entre 30000 et 32767.



```
apiVersion: v1
kind: Service
metadata:
  name: service-frontend
spec:
  selector:
    app: frontend
  type: NodePort
  ports:
    - protocol: TCP
      nodePort: 31234
      port: 8080
      targetPort: 80
```

Kubernetes

Si un Pod est associé à plusieurs répliquas, une répartition de charge (Round Robin) est effectuée par kube-proxy vers les différents répliquas.



Volumes

Kubernetes

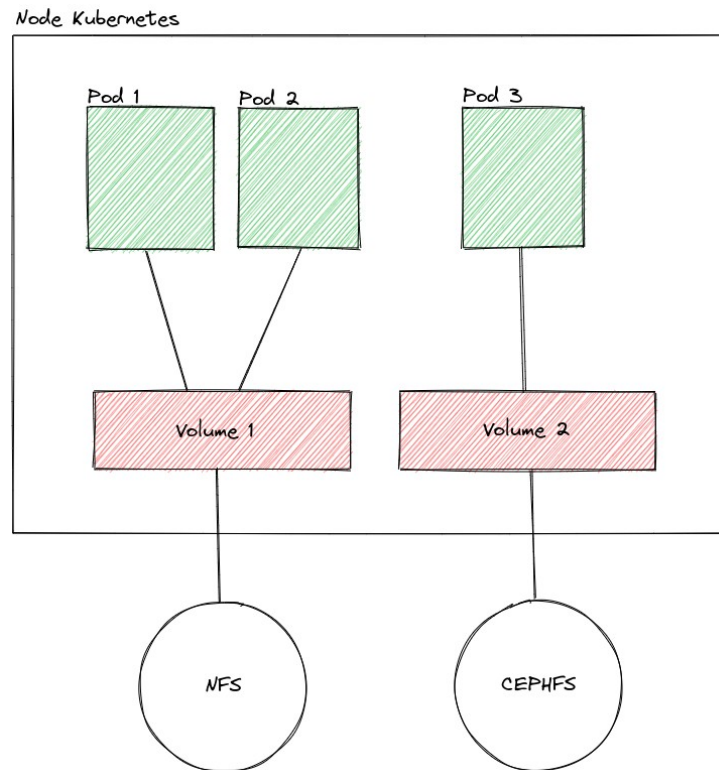
Volumes

Comme pour Docker, Kubernetes nécessite des volumes persistants pour conserver les données même après suppression des Pods.

Kubernetes apporte une gestion plus complète de ces volumes :

- durée de vie
- mode d'accès
- partage d'accès

De nombreux types de volumes existent : **hostPath**, **iscsi**, **nfs**, **cephfs**...



Kubernetes

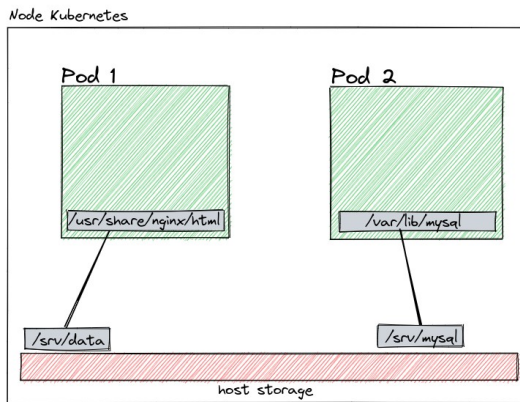
Volumes hostPath

Permet de monter un dossier de l'hôte (node kubernetes) dans le(s) container(s) du Pod.

Si le Pod se déplace sur un autre node, il ne pourra plus accéder aux données du volume : utilisable uniquement sur un Kubernetes mono-node.

Le type pour `hostPath` peut être :

- Directory
- DirectoryOrCreate



```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
  volumeMounts:
  - mountPath: /usr/share/nginx/html
    name: nginx-data
  volumes:
  - name: nginx-data
    hostPath:
      path: /srv/data
      type: Directory
```

Kubernetes

Persistent Volumes

Les Persistent Volumes permettent de déclarer un espace de stockage provisionné statiquement par un administrateur.

Il peuvent utiliser les différents types vus précédemment.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nginx-data
spec:
  storageClassName: localdata
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  reclaimPolicy: recycle
  hostPath:
    path: "/srv/data"
```

Spécifications :

- **capacity** : permet de définir la taille de l'espace de stockage à provisionner.
- **StorageClassName** : utilisé pour sélection par les Claims.
- **accessMode** : permet de définir les types d'accès au stockage
 - ReadWriteOnce : accessible en lecture et écriture sur un seul Pod.
 - ReadOnlyMany : accessible en lecture seule sur plusieurs Pods.
 - ReadWriteMany : accessible en lecture et écriture sur plusieurs Pods.
- **type de PV** : hostPath, nfs...
- **reclaimPolicy** : permet de définir le comportement suite à la suppression d'un PVC lié à un PV
 - delete : permet de supprimer le PV et le stockage associé pour les stockages qui le supportent.
 - recycle : on vide le PV et le PV devient à nouveau accessible pour un PVC.
 - retain : on conserve les données du PV mais pour le réutiliser, il faut supprimer le PV et le recréer.

Persistent Volume Claim

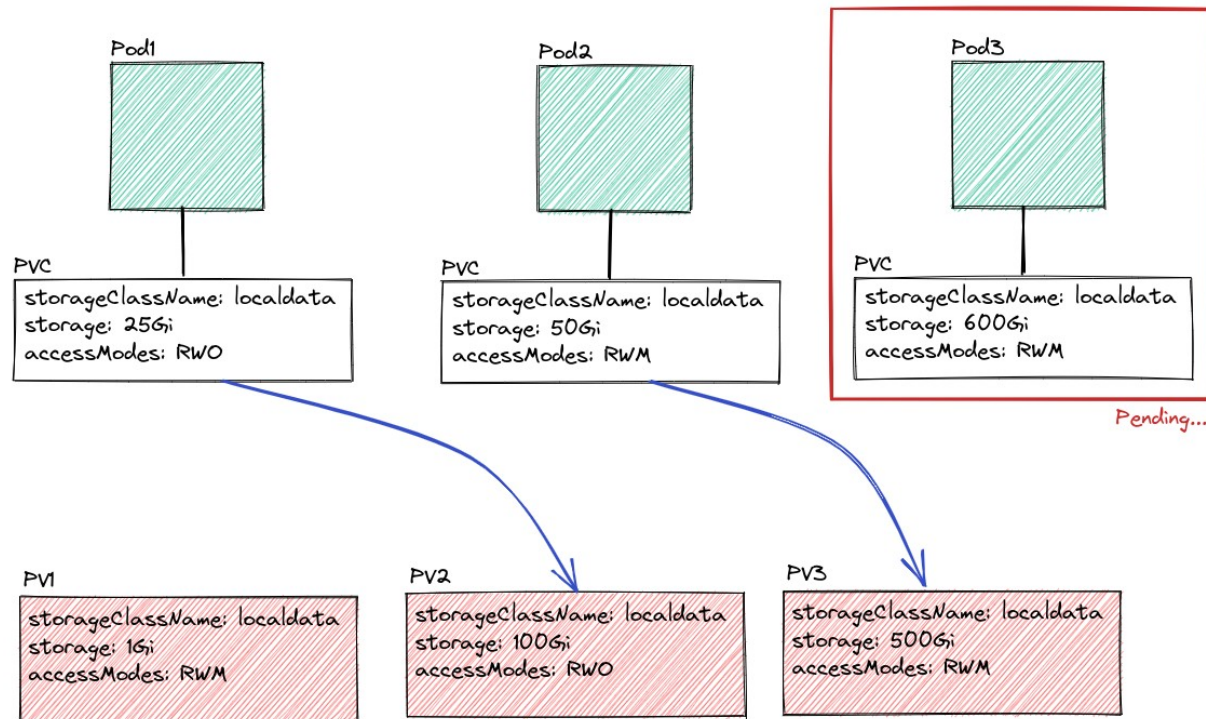
Le PVC permet d'utiliser les ressources d'un PV pour un Pod.

Le PVC permet de chercher un PV qui pourrait convenir aux spécifications.

Si aucun PV ne convient :

- Le PVC et le Pod reste dans l'état **Pending**
- un provisionnement dynamique peut être effectué grâce aux **Storage Class** proposées par le fournisseur Cloud.
- Le PVC va piocher parmi les PV ayant le bon **StorageClassName**.

Persistent Volume Claims



```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nginx-data-pvc
spec:
  storageClassName: localdata
  resources:
    requests:
      storage: 1Gi
  accessModes:
    - ReadWriteOnce
```

Deployments

Kubernetes

Deployment – Déploiements

Un déploiement décrit un état à atteindre. Le contrôleur doit effectuer les modifications pour atteindre cet état (pas forcément un recréation complète).

- Permet de déployer un **ReplicaSet**.
- Possibilité de supprimer le **déploiement**.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

ReplicaSet

Kubernetes

ReplicaSet

Le but des ReplicaSet est de maintenir un certain nombre de répliquas de Pods pour garantir la haute disponibilité.

Ils sont déployés essentiellement par Deployments.

Il est possible de changer le nombre de répliquas en ligne de commande.

```
kubectl scale replicas=4 deployment frontend
```

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: php-redis
          image: gcr.io/google_samples/gb-frontend:v3
```

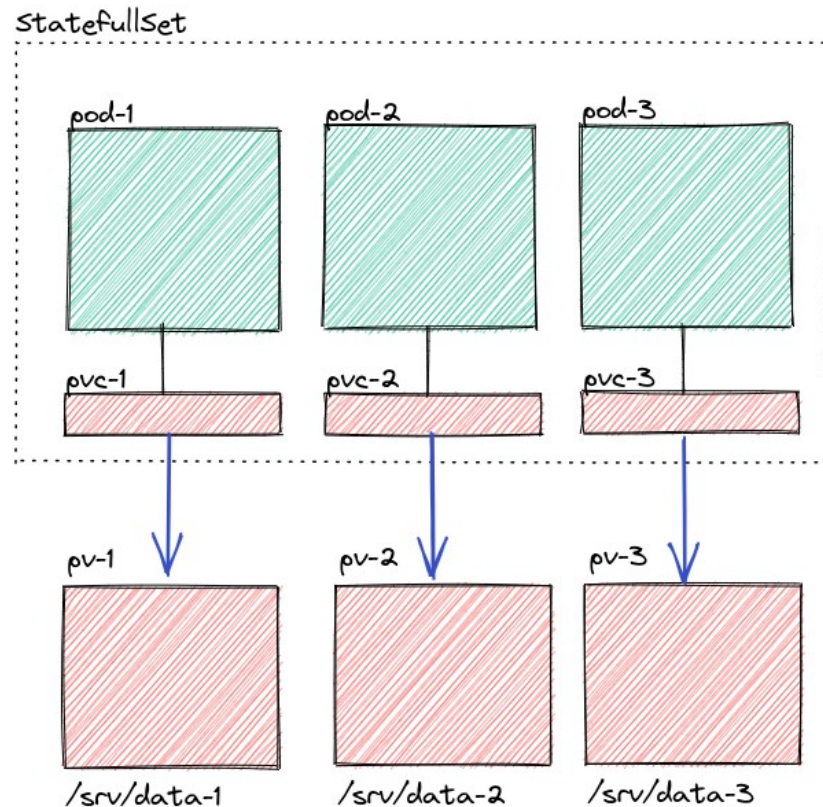
StatefulSet

StatefulSet

Destiné aux applications **Statefull** : applications qui nécessitent de conserver des données pour chaque répliqua

Adapté pour les **clusters de base de données**.

Génère des **PVC** pour chaque Pod du StatefulSet : le stockage est différent pour chaque répliqua.



Kubernetes

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: nginx-sts
spec:
  selector:
    matchLabels:
      app: nginx
  serviceName: nginx-service
  replicas: 4
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: k8s.gcr.io/nginx-slim:0.8
        ports:
        - containerPort: 80
          name: web
        volumeMounts:
        - name: www
          mountPath: /usr/share/nginx/html
```

```
volumeClaimTemplates:
- metadata:
  name: www
  spec:
    accessModes:
    - ReadWriteOnce
    storageClassName: "local-data"
    resources:
      requests:
        storage: 1Gi
```

StatfulSet

Kubernetes

StatefulSet

Nécessite un **service headless** de manière à ne pas utiliser le **Load Balancing** pour accéder aux pods.

Dans les containers

nginx-service renvoie les IP de chaque pod du StatefulSet.

nginx-sts-0.nginx-service renvoie l'IP du pod **nginx-sts-0**.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  labels:
    app: nginx
spec:
  ports:
    - port: 80
      name: web
  clusterIP: None
  selector:
    app: nginx
```

DaemonSet

Kubernetes

DaemonSet

Le DaemonSet permet d'exécuter des Pods sur chaque node du cluster.

Utilisé pour :

- Service de stockage du cluster sur chaque node.
- Service de récupération des logs.
- Service de supervision.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:
    k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
  template:
    metadata:
      labels:
        name: fluentd-elasticsearch
    spec:
      tolerations:
```

Jobs

Kubernetes

Jobs

Les Jobs permettent une exécution ponctuelle d'un ou de plusieurs Pods.

Plusieurs essais sont possibles en cas d'échec et au bout de ceux-ci, le Job est supprimé.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl:5.34.0
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
        restartPolicy: Never
      backoffLimit: 4
```

CronJob

Kubernetes

CronJob

Un CronJob permet d'exécuter des Jobs de manière **périodique**.

Il s'agit de l'équivalent du service **cron**.

Cela peut permettre de d'éviter un **container sidecar** incluant une exécution cron.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox:1.28
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - date; echo Hello from the Kubernetes cluster
          restartPolicy: OnFailure
```

```
# |----- minute (0 - 59)
# |----- hour (0 - 23)
# |----- day of the month (1 - 31)
# |----- month (1 - 12)
# |----- day of the week (0 - 6) (Sunday to Saturday;
# |                                     7 is also Sunday on some systems)
# |                                     OR sun, mon, tue, wed, thu, fri, sat
# |
# * * * * *
```

Exploitation Kubernetes

Labels et Annotations

Kubernetes

Labels et annotations

S'applique sur **toutes les ressources** kubernetes.

Les **labels** servent à identifier et sélectionner des ressources.

Les **annotations** ne servent qu'à donner des indications sur la ressources mais ne servent pas à les sélectionner.

Peuvent s'utiliser dans de nombreux contextes :

- Dev, Prod
- Nom de l'application
- Type de service
- Localisation géographique
- ...

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    env: prod
    group: front
    name: nginx-1
spec:
  containers:
    - name: nginx
      image: nginx
```

Kubernetes

Labels et Annotations

Utilisation des **selector** pour sélectionner les ressources en fonction de leur label.

```
kubectl get pods --show-labels
kubectl get pods --label-columns=env
kubectl get pods --selector="env=dev"
```

```
env=dev
env!=dev
env in (dev,prod)
env notin (dev,prod)
```

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    env: prod
    group: front
    name: nginx-1
spec:
  containers:
    - name: nginx
      image: nginx
---
apiVersion: v1
kind: Pod
metadata:
  labels:
    env: dev
    group: front
    name: nginx-2
spec:
  containers:
    - name: nginx
      image: nginx
```

Affinité - Anti-affinité

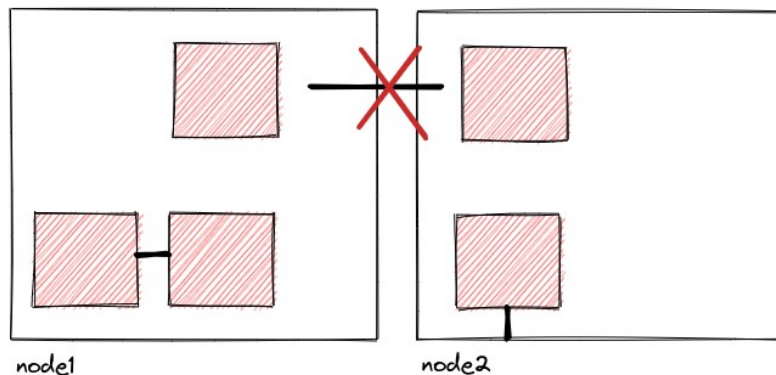
Kubernetes

L'affinité peut concerner :

- Les **Nodes** : les Pods doivent être associés à un ou des nodes suivants des règles définies.
- Les **Pods** : les Pods doivent s'exécuter sur un même node.

L'antiaffinité peut concerner :

- Les **Pods** : les pods ne doivent pas s'exécuter sur le même node.



Affinité - Labels sur les nodes

Les nodes ont des labels assignés par défaut.

Ces labels et ceux définis manuellement peuvent permettre de définir des règles d'affinité.

```
# Affichage des labels
kubectl get nodes --show-labels

# Application d'un label
kubectl label nodes knode1 disktype=ssd

#Suppression d'un label
kubectl label nodes knode1 disktype-
```

Affinité sur les nodes

`requiredDuringSchedulingIgnoredDuringExecution`

→ le pod devra obligatoirement être positionné sur un node ayant ce label.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disktype
                operator: In
                values:
                  - ssd
```

Kubernetes

Affinité sur les nodes

`preferredDuringSchedulingIgnoredDuringExecution`

→ le pod préférera être positionné sur un node ayant ce label mais s'il n'y en a pas de disponible ou ayant suffisamment de ressource, il se positionnera sur un autre.

Opérateurs

Les mêmes que sur les `selector: In, NotIn, Exists, DoesNotExist, Gt, Lt`

Weight

Dans le cas du `preferredDuringScheduling`, permet de préciser les préférences dans les différentes règles de sélection des nodes.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 1
          preference:
            matchExpressions:
              - key: disktype
                operator: In
                values:
                  - ssd
```

Affinité – Anti-affinité inter-pods

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    env: dev
    group: front
  name: nginx-2
spec:
  containers:
  - name: nginx
    image: nginx
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: group
            operator: In
            values:
            - front
        topologyKey: kubernetes.io/hostname
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    env: dev
    group: front
  name: nginx-2
spec:
  containers:
  - name: nginx
    image: nginx
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: group
            operator: In
            values:
            - front
        topologyKey: kubernetes.io/hostname
```

Health-Checks

Liveness - Readiness

Kubernetes – Health Checks

Objectifs

Gérer le cas où des applications doivent être redémarrées

Détecter les images problématiques

Éviter les coupures de service en cas de déploiement

kubectl describe pod permet de visualiser les checks effectués.

Vérifications

- **command** (exec) : exécution d'une commande
- **http** (httpGet) : utilisation d'une requête HTTP
- **tcp** (tcpSocket) : connexion à un port TCP

C'est le service kubelet qui s'occupe de ces tests.

Kubernetes – Health Checks

Types de vérifications

- **Liveness** : Permet de redémarrer automatiquement des pods si leur état semble problématique.
- **Readiness** : Permet de remplacer des pods défectueux.

Les deux types peuvent être utilisés sur le même Pod.

Variables

- **initialDelaySeconds** : durée au bout de laquelle est effectuée le premier test. Un Pod peut mettre du temps à démarrer.
- **periodSeconds** : fréquence de vérification
- **timeoutSeconds** : durée au bout de laquelle on considère qu'il y a un échec.
- **successThreshold** : nombre de vérifications pour réactiver le Pod.
- **failurethreshold** : nombre de vérifications pour considérer un échec.

Kubernetes – Health Checks

Vérification par une commande

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-exec
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/busybox
    args:
      - /bin/sh
      - -c
      - touch /tmp/healthy; sleep 30; rm -f /tmp/healthy; sleep 600
    livenessProbe:
      exec:
        command:
          - cat
          - /tmp/healthy
      initialDelaySeconds: 5
      periodSeconds: 5
```

Kubernetes – Health Checks

Vérification par une requête HTTP

Paramètres :

- Path
- Port
- Host
- Scheme

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness-http
    name: liveness-http
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
      volumeMounts:
        - mountPath: /usr/share/nginx/html
          name: nginx-data
      livenessProbe:
        httpGet:
          path: /health.test
          port: 80
          initialDelaySeconds: 5
          periodSeconds: 5
      volumes:
        - name: nginx-data
          hostPath:
            path: /srv/data
```

Kubernetes – Health Checks

Vérification par connexion TCP

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness-http
    name: liveness-http
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
      livenessProbe:
        tcpSocket:
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 5
```

Configmap

Configmap

Une Configmap est un objet permettant de stocker des ensembles de clé / valeurs non confidentielles.

Elle peut être injecté dans un pod comme

- variables d'environnement,
- arguments d'une ligne de commande
- fichier de configuration.

Cela permet d'éviter de stocker une configuration en dur dans l'image du container et de pouvoir modifier facilement la configuration.

Kubernetes

Création d'une configmap

Utilisation d'un nom

Structuration dans data

Peut prendre le format d'un fichier

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  # property-like keys; each key maps to a simple value
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"

  # file-like keys
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
  user-interface.properties: |
    color.good=purple
    color.bad=yellow
    allow.textmode=true
```

Kubernetes

Utilisation d'une configmap en tant que variables d'environnement et de fichiers

Les configmaps montés dans le Pod sont mis à jour automatiquement.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  # property-like keys; each key maps to a simple value
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"

  # file-like keys
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
  user-interface.properties: |
    color.good=purple
    color.bad=yellow
    allow.textmode=true
```

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-demo-pod
spec:
  containers:
    - name: demo
      image: alpine
      command: ["sleep", "3600"]
      env:
        - name: PLAYER_INITIAL_LIVES
          valueFrom:
            configMapKeyRef:
              name: game-demo
              key: player_initial_lives
        - name: UI_PROPERTIES_FILE_NAME
          valueFrom:
            configMapKeyRef:
              name: game-demo
              key: ui_properties_file_name
      volumeMounts:
        - name: config
          mountPath: "/config"
          readOnly: true
  volumes:
    - name: config
      configMap:
        name: game-demo
        items:
          - key: "game.properties"
            path: "game.properties"
          - key: "user-interface.properties"
            path: "user-interface.properties"
```

Secrets

Principe

Les Secrets sont des objets qui contiennent des données sensibles comme des mots de passe, des tokens...

Ils permettent d'éviter de placer ces secrets dans le code applicatif.

Ils sont équivalents aux **configmaps** mais en intégrant des données sensibles.

Utilisation des secrets

- Sous forme de variables d'environnements
- Sous forme de fichiers
- Pendant la récupération des images

Kubernetes

Gestion des secrets avec kubectl

```
# Directement dans la ligne de commande
kubectl create secret generic db-access --from-literal=user=root --from-literal=password='P@ssw0rd'

# A partir de fichiers
echo -n 'root' | base64 > ./username.txt
echo -n 'P@ssw0rd' | base64 > ./password.txt
kubectl create secret generic db-access --from-file=./username.txt --from-file=./password.txt

# Modification des secrets (en effectuer en base64)
kubectl edit secrets db-access

# Suppression d'un secret
kubectl delete secret db-access

# Affichage des secrets (en base64)
kubectl get secret db-access -o json
```

```
{
  "apiVersion": "v1",
  "data": {
    "password": "UEBzc3cwcmQ=",
    "user": "cm9vdA=="
  },
  "kind": "Secret",
  "metadata": {
    "creationTimestamp": "2022-11-30T22:40:14Z",
    "name": "db-access",
    "namespace": "default",
    "resourceVersion": "15695",
    "uid": "774e8ff1-b0e5-4841-b274-b9f13603072f"
  },
  "type": "Opaque"
}
```

```
echo -n 'UEBzc3cwcmQ=' | base64 --decode
# retourne : P@ssw0rd
```

Kubernetes

Gestion des secrets avec un fichier de configuration

```
echo -n 'root' | base64  
echo -n 'P@ssw0rd' | base64
```

```
apiVersion: v1  
kind: Secret  
metadata:  
  name: db-access  
type: Opaque  
data:  
  username: cm9vdA==  
  password: UEBzc3cwcmQ=
```

Kubernetes

Utilisation dans un Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-secret
spec:
  containers:
    - name: nginx
      image: nginx
      command:
        - "bash"
        - "-c"
        - "sleep 6000"
      env:
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: db-access
              key: username
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-access
              key: password
      volumeMounts:
        - name: secrets
          mountPath: "/etc/secrets"
          readOnly: true
  volumes:
    - name: secrets
      secret:
        secretName: db-access
        optional: false
        defaultMode: 0400
```

Kubernetes en production

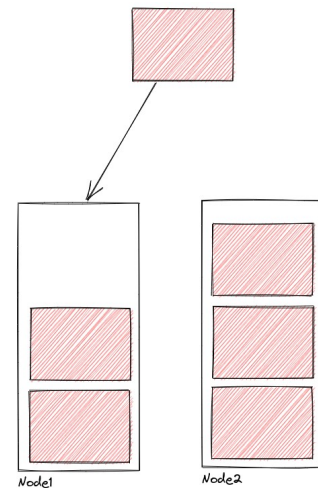
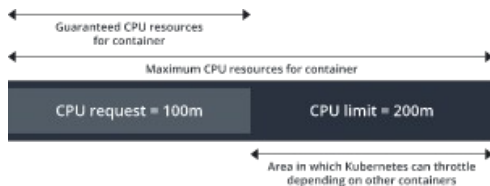
Limitation des ressources - Requests et Limits

Kubernetes

Limitation des ressources

Kubernetes permet de gérer les ressources pour permettre une meilleure cohabitation des containers ainsi qu'une optimisation du placement des pods.

- **request** : le container doit pouvoir utiliser cette ressource. Kubernetes va placer le Pod sur un node disposant de cette ressource.
- **limit** : le container ne peut pas utiliser plus que cette ressource.



Kubernetes

Limitation des ressources

Types de ressources :

- **cpu** : exprimé en % et m (milliCPU)
- **mem** : exprimé en Mi (mebibytes) ou M (megabytes), k, G, T, P, T, Ki, Gi, Ti, Pi, Ei

```
apiVersion: v1
kind: Pod
metadata:
  name: frontend
spec:
  containers:
  - name: app
    image: images.my-company.example/app:v4
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
  - name: log-aggregator
    image: images.my-company.example/log-aggregator:v6
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```


Namespaces, contextes et quotas

Kubernetes

Namespaces

Les namespaces permettent de cloisonner les objets.

```
kubectl get namespaces  
kubectl create namespace cepn
```

Les pods peuvent avoir des noms identiques sur des namespaces différents.

kube-system et default sont des namespaces créés par défaut.

Contextes

Permet de définir quel utilisateur et sur quel cluster nous travaillons.

```
kubectl config get-contexts  
kubectl config set-context cepn --namespace cepn --user kubernetes-admin --cluster kubernetes  
kubectl config use-context cepn
```

Kubernetes

Quotas de namespaces

Les quotas de Namespaces permettent de réserver et limiter les ressources sur les différents Namespaces. Cela permet d'éviter qu'une équipe consomme toutes les ressources du Cluster.

Nombre de ressources pouvant être limitées :

- configmaps
- PersistentVolumeClaims
- Pods
- Services
- Nodeports
- Secrets

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: test-quota
  namespace: quota-demo
spec:
  hard:
    requests.cpu: "200m"
    limits.cpu: "300m"
    pods: "10"
```

```
kubectl describe resourcequota/test-quota --namespace quota-demo
```

HPA – Horizontal Pod Autoscaling

Autoscaling

- HPA : adaptation du nombre de Pods en fonction de la charge.
- VPA : adaptation des ressources du Pod.
- Cluster autoscaling : adaptation du nombre de nodes.

Seule l'HPA sera traitée dans ce document.

Kubernetes

HPA – Horizontal Pod Autoscaling

S'appuie sur la consommation des ressources :

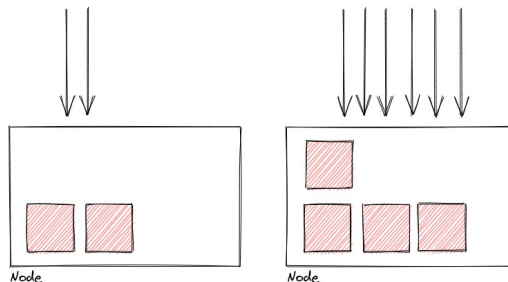
- CPU
- Mémoire

Nécessité d'installer un serveur de métriques

```
# Préférer télécharger le fichier et ajouter --kubelet-insecure-tls au deployment de metric-server  
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Test de montée en charge

```
ab -n 100000 -c 100 http://192.168.179.116:31000/
```



```
apiVersion: autoscaling/v2  
kind: HorizontalPodAutoscaler  
metadata:  
  name: hpa-nginx  
spec:  
  scaleTargetRef:  
    apiVersion: apps/v1  
    kind: Deployment  
    name: nginx-deployment  
  minReplicas: 1  
  maxReplicas: 10  
  metrics:  
    - type: Resource  
      resource:  
        name: cpu  
        target:  
          type: Utilization  
          averageUtilization: 10
```

Maintenance

drain - cordon

Kubernetes

Maintenance

Kubernetes permet de facilement libérer des nodes pour en permettre la maintenance.

Cordon

`kubectl cordon node1` permet de stopper la planification de ressource sur un node (node1 ici).

`kube-scheduler` n'affectera pas de nouveaux Pods sur le node concerné.

Les Pods déjà positionnés sur ce node **resteront actifs** et ne seront **pas supprimés**.

`kubectl uncordon node1` permet d'autoriser à nouveau la planification de Pods sur le node.

Drain

`kubectl drain node1` permet de supprimer tous les pods du node concerné.

Les pods en question seront **recréés** sur un autre node dans le cas d'un **deployment**.

RBAC – Gestion des accès

Kubernetes

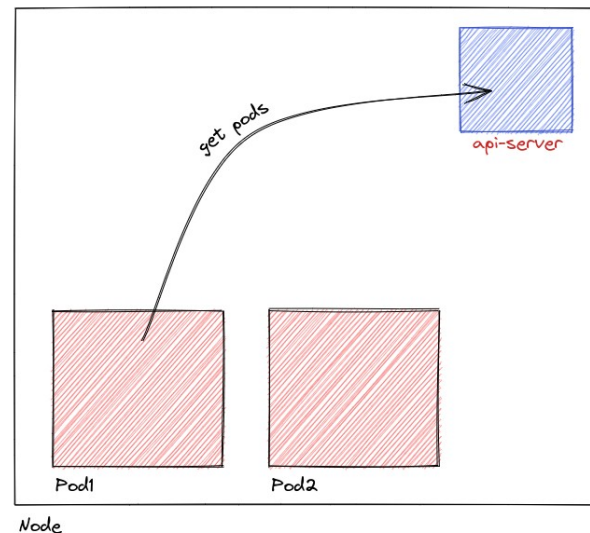
RBAC – Gestion des accès

Les comptes de service permettent à des processus à l'intérieur des containers d'avoir accès à l'API Server.

service account : utilisateur applicatif

cluster role : profil permettant de définir les accès sur les ressources

cluster role binding : association entre le cluster role et le service account



Kubernetes

Cluster Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cr1
rules:
  - apiGroups:
    - ""
    resources:
    - "configmaps"
    verbs:
    - "get"
    - "list"
    - "watch"
    - "patch"

  - apiGroups:
    - ""
    resources:
    - "pods"
    verbs:
    - "describe"
```

Service Account

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: anthony
```

Cluster Role Binding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: crb1
subjects:
- kind: ServiceAccount
  name: anthony
  namespace: default
roleRef:
  kind: ClusterRole
  name: cr1
  apiGroup: rbac.authorization.k8s.io
```

Kubernetes

Il faut créer un token pour l'utilisation du Service Account.

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-sa-anthony
  annotations:
    kubernetes.io/service-account.name: "anthony"
type: kubernetes.io/service-account-token
```

```
kubectl get secrets secret-sa-anthony -o yaml
```

Kubernetes

Utilisation dans un Pod

Il faut monter le secret contenant le token pour l'utiliser dans les requêtes à l'API.

```
TOKEN=$(cat /var/run/secrets/kubernetes.io/secretaccount/token)
curl -H "Authorization: Bearer $TOKEN" https://kubernetes/api/v1/pods --insecure
```

```
apiVersion: v1
kind: Pod
metadata:
  name: alpine
spec:
  serviceAccountName: anthony
  containers:
  - name: alpine
    image: alpine
    command:
    - "sh"
    - "-c"
    - "sleep 3600"
  resources:
    limits:
      cpu: "500m"
      memory: "128Mi"
  volumeMounts:
  - mountPath: /var/run/secrets/kubernetes.io/secretaccount
    name: secret-sa-anthony
  volumes:
  - name: secret-sa-anthony
    secret:
      secretName: secret-sa-anthony
```

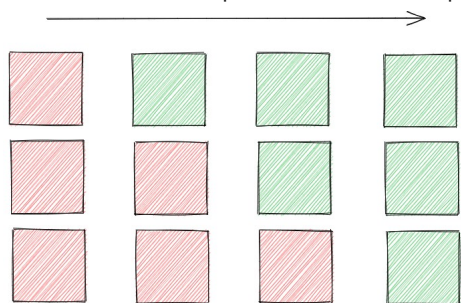
Stratégie de mise à jour sans interruption

Kubernetes

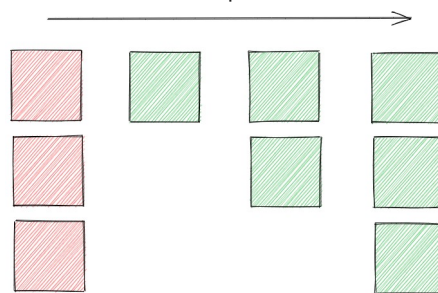
Elles s'utilisent dans les **deployments** et permettent de définir comment sont déployées les mises à jour des containers.

Deux stratégies :

- **Rolling updates** : les Pods sont remplacés un à un sans coupure de service. C'est le comportement par défaut ;
- **Recreate** : tous les pods sont supprimés et recréés en suivant. Une coupure de service a lieu.



Rolling update



Recreate

Kubernetes

Rolling update

maxSurge : autorise des pods en plus de ce qui est prévu dans le déploiement.

maxUnavailable : autorise un certain nombre de pods down donc un total inférieur au nombre prévu pendant la mise à jour.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.20
          resources:
            limits:
              memory: "128Mi"
              cpu: "500m"
          ports:
            - containerPort: 80
```


Kubernetes

Recreate

Les Pods sont tous supprimés et les Pods de la nouvelle version sont créés en suivant.

Une coupure de service intervient pendant la mise à jour.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 5
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.20
          resources:
            limits:
              memory: "128Mi"
              cpu: "500m"
          ports:
            - containerPort: 80
```

Kubernetes

Historique et rollout

Les annotations permettent de donner des informations sur les précédentes mises à jour.

```
template:
  metadata:
    annotations:
      kubernetes.io/change-cause: "Mise à jour vers la version 1.20"
```

```
# Etat de la mise à jour
kubectl rollout status deployment nginx-deployment

# Mise en pause de la mise à jour
kubectl rollout pause deployment nginx-deployment

# Reprise de la mise à jour
kubectl rollout resume deployment nginx-deployment

# Historique des mises à jour (penser à placer une annotation)
kubectl rollout history deployment nginx-deployment --revision 1

# revenir en arrière
kubectl rollout undo deployment nginx-deployment --to-revision=3
```

Ingress

Kubernetes

Ingress

Ingress est un objet Kubernetes permettant de gérer les accès depuis l'extérieur (HTTP).

Il permet d'éviter de publier des nodePorts pour chaque service accessible depuis l'extérieur et simplifie la configuration du reverse proxy frontal.

Il peut gérer :

- la répartition de charge
- TLS
- les virtual hosts basés sur les noms



kubernetes

Contrôleurs Ingress

Il faut utiliser un ingress controller pour que les ressources ingress puissent être opérationnelles.

Contrôleurs Ingress disponibles :

- Nginx
- AWS
- GCE
- HAProxy
- Istio
- Traefik
- ...

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webservers
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
      - path: /web-1
        backend:
          service:
            name: service-web-1
            port: "80"
    - host: "web-2.avalone-fr.com"
      http:
        paths:
        - path: /
          backend:
            service:
              name: service-web-2
              port: "80"
```

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.5.1/deploy/static/provider/cloud/deploy.yaml
```